Applicability of Non-Asymptotic Optimizations in Range Filters

Andrei Ogurtsov

April 30, 2026

Abstract

We implement and evaluate Approximate Range Emptiness (ARE) filters for LSM-tree storage systems. Starting from the information-theoretically optimal construction of Goswami et al. (SODA 2015), we develop a family of practical filters that explore different hash function designs: TODO

Contents

Chapter 1

Introduction

1.1 Notation

General. We identify L-bit keys with the integers $\{0, 1, \ldots, 2^L - 1\}$ via the standard binary representation. Lexicographic comparison of bit strings of the same length coincides with numerical comparison of the corresponding integers, so we use the integer view throughout (this is the natural formulation when the construction performs arithmetic such as rotations or reductions modulo a power of two).

Sets and parameters.

L	Key length in bits.
$U = [0, 2^L)$	Universe of L-bit keys.
n	Number of input keys, $n = \lvert S \rvert$.
$S \subseteq U$	The set of keys in a Range Filter, $\lvert S \rvert = n$.
$Y = U \setminus S$	Non-keys (all points not in S).
$\mathcal{L}$	Maximum query range length.
ε	Target false positive rate.

Hash function and fingerprints.

h	Hash function $h\colon U \to U'$.
K	Fingerprint length in bits, $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$.
$U' = [0, 2^K)$	Mapped universe.
$r = 2^K$	Mapped universe size.
$S' = h(S)$	Fingerprints of the input keys.
$Y' = h(Y)$	Images of non-keys under h.

Metrics.

FPR	False Positive Rate.
BPK	Bits Per Key: total filter size divided by n.

Structures.

ERE	Exact Range Emptiness — zero-error range filter.
ARE	Approximate Range Emptiness — range filter with one-sided error $\leq \varepsilon$.

1.2 Problem Statement

Let $U = [0, 2^L)$ be the universe of L-bit keys (Section 1.1). Given a set $S \subseteq U$ of n keys, a maximum query range length $\mathcal{L}$, and a target false positive rate ε, the *approximate range emptiness problem* asks to build a data structure that answers "$[a, b] \cap S = \emptyset$?" for intervals of length $\leq \mathcal{L}$, with:

- **No false negatives:** if $[a, b] \cap S \neq \emptyset$, the answer is always "not empty".
- **Bounded false positives:** if $[a, b] \cap S = \emptyset$, the answer is "not empty" with probability at most ε.

Let $Y = U \setminus S$ denote all keys not in the filter.

1.2.1 Information-Theoretic Bounds

Goswami et al. [2015] establish the following bounds.

Theorem 1 (Lower bound, Goswami et al. [2015, §2]). *Any data structure solving the range emptiness problem requires at least $n\big(\log_2(\mathcal{L}/\varepsilon) - c\big)$ bits for an absolute constant $c > 0$.*

Remark 1. *The lower bound is worst-case over the input: it holds for every set $S \subseteq U$ with $|S| = n$, with no assumptions on the structure or distribution of S.*

Remark 2. *In practice, real datasets are often more regular. If the input is known to have additional structure, a data structure may use that structure to reduce the required space below the general bound.*

Theorem 2 (Upper bound, Goswami et al. [2015, §3]). *The lower bound of Theorem 1 is achieved up to an additive $O(n)$ term via two layers:*

1. *A locality-preserving hash (Section 4.1) $h\colon U \to U'$, where $U' = [0, 2^K)$ and $r = |U'| = 2^K$. The hash partitions U into blocks of size r, applies an independent rotation within each block, and then maps the shifted block onto U' by translation. The hash is drawn from a pairwise-independent family, so for any two distinct keys $x_1, x_2 \in U$, $\Pr[h(x_1) = h(x_2)] \leq 1/2^K$.*
2. *An **Exact Range Emptiness (ERE)** (Section 2.1) structure over $S' = h(S) \subseteq U'$: a succinct structure with zero error and space $n \log_2(r/n) + O(n)$ bits, which answers "$[a', b'] \cap S' = \emptyset$?".*

The resulting space is $n \log_2(\mathcal{L}/\varepsilon) + O(n)$ bits, or equivalently $\log_2(\mathcal{L}/\varepsilon) + O(1)$ bits per key — matching the lower bound.

The fingerprint length $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$ controls accuracy: because $\Pr[h(x_1) = h(x_2)] \leq 1/2^K$ for any two distinct keys, the overall error probability of the construction is bounded by $\varepsilon \leq n\mathcal{L}/2^K$. Thus, increasing K by one bit halves the false positive rate. After ERE compression (which subtracts $\log_2 n$ for block indexing), the effective cost per key is $K - \log_2 n = \log_2(\mathcal{L}/\varepsilon) + O(1)$

1.2.2 The Role of the Hash Function

The hash h projects U down to U'. Under this projection, the keys S map to fingerprints $S' = h(S)$, and the non-keys $Y = U \setminus S$ map to $Y' = h(Y)$.

A false positive occurs when Y' overlaps with S': a query point $y \in Y$ has $h(y) \in S'$, so the structure cannot distinguish it from a real key. We call the pre-images of a fingerprint $x' \in S'$ its **phantom points** — all keys y such that $h(y) = x'$. The fewer phantoms overlap with Y', the lower the FPR.

The ideal hash would map S' and Y' to disjoint regions of U' — zero overlap, zero false positives. In practice this is impossible within the space budget, but the choice of h determines how well S' and Y' separate for a given data distribution. This motivates two directions explored in this work: alternative hash function designs (Chapter 4) and data-adaptive segmentation strategies that bypass hashing entirely on dense regions (Chapter 5).

Chapter 2

Exact Range Emptiness

2.1 Succinct Representation (SODA 2015, §3.2)

The inner layer of every approximate filter is an *Exact Range Emptiness* (ERE) structure: given a universe U' of K-bit keys and a set $S' \subseteq U'$ of n keys, it answers "$[a, b] \cap S' = \emptyset$?" exactly (no false positives or false negatives).

Remark 3. *Throughout this chapter, S' denotes the input to ERE; in the ARE pipeline (Chapter 4), $S' = h(S)$.*

2.1.1 High-Level Structure

To achieve the information-theoretic minimum of $n \cdot (K - \log_2(n) + O(1)) = n \log_2(\frac{r}{n}) + O(n)$ bits, the structure divides the universe U' into $B = 2^{\lfloor \log_2 n \rfloor}$ equal-sized blocks. Conceptually, ERE has two levels: a succinct navigation layer over blocks, and a compact local representation of the keys.

All keys inside one block have common $\log_2 B = \lfloor \log_2 n \rfloor$ bits prefix. Therefore, key suffixes can be stored in the array with $w = K - \lfloor \log_2 n \rfloor$ bits per key. The succinct navigation layer is used to determine block boundaries.

2.1.2 Original ERE Structure

We now describe the original ERE construction of Goswami et al. [2015] more precisely. It consists of the following parts:

1. D_1 succinct bit vector: stores block occupancy: $D_1[i] = 1$ iff block i contains at least one key.

2. D_2 succinct bit vector: encodes the sizes of non-empty blocks in unary form. If a non-empty block i contains n_i keys, then it contributes a 1 followed by n_i zeros.

3. A_{ds} the array of structures: each data structure consists of:

   The sorted list of suffixes

   A Weak Prefix Search index over those suffixes (Section 2.1.6).

A query range is split into at most three parts: a sequence of complete blocks and up to two partially covered boundary blocks. The complete-block part is answered by the global succinct layer using D_1 and D_2. The boundary parts are answered by locating the corresponding local structure in A_{ds} via D_1 and D_2, and then querying that local structure inside the block; the local emptiness check itself reduces to two weak prefix queries (Section 2.1.6).

2.1.3 Local Search in Buckets

The original paper [Goswami et al., 2015] suggests a Weak Prefix Search structure (Hollow Z-Fast Trie) for $O(1)$ worst-case local queries. We use binary search on bit-packed suffixes, with an adaptive fallback to linear scan for small buckets instead. The reason is practical. Binary search and linear scan are faster in practice than accessing an additional $O(1)$ indexing structure with a big hidden constant, while also avoiding its memory overhead. Details are given in Section 2.1.6.

2.1.4 One-Vector Optimization

In the optimized variant, we simplify the navigation layer. Instead of storing occupancy and non-empty block offsets separately in D_1 and D_2, we replace them with a single succinct bit-vector D. Every block i with n_i keys is encoded as a single 1 followed by n_i zeros, whether the block is empty or not. In particular, an empty block (with $n_i = 0$) is encoded by a single 1. One final sentinel 1 is appended at the end.

Effect on space and navigation. Let M denote the number of non-empty blocks. In the original layout, the metadata size is

$$|D_1| + |D_2| = B + (n + M + 1).$$

In the optimized layout, the metadata becomes

$$|D| = B + n + 1.$$

The exact metadata reduction is M bits. Operationally, this means that the old two-stage lookup through D_1 and D_2 is replaced by direct navigation via SELECT on D.

Metadata footprint on uniform input. Table 2.1 reports the metadata BPK on n uniform 64-bit keys for $n \in \{2^{20},\ 2^{24},\ 2^{28}\}$. The information-theoretic delta is ∼0.63 BPK, matching the analytic prediction $M/n \to 1 - e^{-1} \approx 0.632$ at $\lambda = 1$ (each block is independently empty with probability e^{-1}, Section 2.1.5). The physical delta is ∼0.80 BPK, amplified by a constant ∼26% overhead from the rank/select index — intrinsic to succinct bit vectors and required for the $O(1)$ navigation guarantees, not a Go allocation artefact. Both deltas are stable across n.

	Num (logical bits)		**Alloc** (with rank/select index)	
n	two-vector	one-vector	two-vector	one-vector
2^{20}	2.63	2.00	3.33	2.53
2^{24}	2.63	2.00	3.33	2.53
2^{28}	2.63	2.00	3.33	2.53
Δ	−0.63 BPK (−24%)		−0.80 BPK (−24%)	

Table 2.1: Metadata BPK for the two-vector (D_1, D_2) layout (Section 2.1.2) versus the one-vector D layout, on n uniform 64-bit keys. *Num*: logical bitvector length (D_1.NUM + D_2.NUM vs. D.NUM); matches the analytic $|D_1|+|D_2| = B+(n+M+1)$ and $|D| = B+n+1$ formulas above. *Alloc*: physical RSDic footprint (bits plus pointer, rank, and select indices). The suffix array A_{ds} is excluded — identical in both layouts.

Against the ∼ 20 BPK total ERE footprint — dominated by the suffix array A_{ds}, which is identical between the two layouts — the absolute saving of 0.80 BPK looks small. But it is a ∼ 24% reduction in metadata: the only part that can shrink without information loss. Cross-distribution measurements are deferred to Chapter 6 (Table 6.2).

Effect on query operation count. The single-vector layout does not strictly reduce the number of RANK/SELECT calls in every case: the original two-vector layout could short-circuit empty boundary blocks via a D_1.BIT probe, while the new layout always pays at least two SELECT(D) per boundary block. However, in the two most common ARE query patterns — short ranges that hit a single non-empty block, or that span two adjacent non-empty blocks — the new layout saves *one* and *three* RANK/SELECT calls per query, respectively, with long ranges that scan both boundary buckets saving up to four. The structural advantage is that every RANK/SELECT now hits one contiguous succinct bit vector D instead of alternating between D_1 and D_2, so the auxiliary index pages and cache lines loaded by the first call are likely to serve subsequent calls in the same query. The full per-case breakdown is given in Table 2.2; end-to-end query latency on uniform and clustered data is reported in Chapter 6 (Table 6.3).

Query type	(D_1, D_2) ops	D ops	Δ	Note
Same block, non-empty	$1\,R + 2\,S = 3$	$2\,S = 2$	$\mathbf{-1}$	**Most frequent, hot path**
Same block, empty	0	$2\,S = 2$	$+2$	Cold-path regression
Adjacent, both non-empty	$2\,R + 4\,S = 6$	$3\,S = 3$	$\mathbf{-3}$	**Most frequent, hot path**
Adjacent, both empty	0	$3\,S = 3$	$+3$	Cold-path regression
Long range, both boundaries non-empty	$4\,R + 4\,S = 8$	$4\,S = 4$	-4	Op count halved
Long range, intermediate non-empty	$2\,R = 2$	$4\,S = 4$	$+2$	More ops, single vector
Long range, all empty	$2\,R = 2$	$4\,S = 4$	$+2$	Cold-path regression

Table 2.2: Number of RANK/SELECT operations per query case for the original two-vector (D_1, D_2) layout (Section 2.1.2) versus the one-vector D layout (Section 2.1.4). R denotes RANK on D_1; S denotes SELECT on D_2 or D as appropriate. Bolded rows are the dominant query patterns in typical ARE workloads. BIT probes and bucket-scan operations are not counted (identical between layouts).

2.1.5 Block sizes and Lookup strategy

Block occupancy under uniform distribution. The Exact Range Emptiness structure divides the universe into $B = 2^{\lfloor \log_2 n \rfloor}$ blocks; each non-empty block stores its keys' suffixes in a packed array (a *bucket*). The expected number of keys per block is $\lambda = n/B \in [1, 2)$. When n is a power of two, $B = n$ and $\lambda = 1$ exactly. Each key falls into a uniformly random block, so the occupancy of a fixed block follows $\mathrm{Bin}(n, 1/B)$, which converges to $\mathrm{Poisson}(\lambda)$ as $n \to \infty$ [Mitzenmacher and Upfal, 2017, Ch. 5].

For $\lambda = 1$, the fraction of empty blocks converges to $e^{-1} \approx 36.8\%$, and the mean occupancy among non-empty blocks to $1/(1 - e^{-1}) \approx 1.58$.

The classical balls-into-bins analysis [Mitzenmacher and Upfal, 2017, Lemma 5.1] bounds the maximum load by $3 \ln n / \ln \ln n$ with probability $\geq 1 - 1/n$. At $n = 2^{30}$ this gives ≈ 21 keys per bucket. Table 2.3 reports the empirical maxima alongside the Poisson reference k^* — the smallest k with $B \cdot \Pr[\mathrm{Poisson}(1) \geq k] < 1$; all measured maxima are well below the Lemma 5.1 bound. Each observed max is the largest bucket over a single realisation of n uniform random 64-bit keys, taken across all B blocks.

Table 2.4 shows how rapidly the Poisson tail drops with k: even at $B = 2^{30}$, the expected number of blocks with ≥ 20 keys is below 10^{-10}, making buckets larger than ~20 keys virtually impossible for uniform data.

n	empty (%)	avg load*	observed max	k^*	$B \cdot \Pr[X \geq k^*]$
2^{20}	36.75	1.58	9	10	0.12
2^{24}	36.79	1.58	9	11	0.17
2^{27}	36.79	1.58	10	12	0.11
2^{30}	36.79	1.58	13	12	0.89

Table 2.3: Bucket statistics for n keys in $B = n$ bins ($\lambda = 1$). *Averaged over non-empty bins. k^*: smallest k with $B \cdot \Pr[\text{Poisson}(1) \geq k] < 1$.

k	$\Pr[\text{Poisson}(1) \geq k]$	expected bins
10	1.1×10^{-7}	1.2×10^{2}
13	6.4×10^{-11}	6.8×10^{-2}
15	3.0×10^{-13}	3.2×10^{-4}
20	1.6×10^{-19}	1.7×10^{-10}
25	2.5×10^{-26}	2.6×10^{-17}

Table 2.4: Poisson tail for bucket occupancy ($\lambda = 1$), $n = B = 2^{30}$.

Non-uniform distributions and worst-case bounds For uniform data the previous paragraph showed that bucket sizes concentrate around $\lambda = 1$: empirical maxima stay below $\sim$13 keys even at $n = 2^{30}$ (Table 2.3), regardless of n.

Real-world data is rarely uniform. The original ARE construction [Goswami et al., 2015] uses a *locality-preserving* hash: consecutive input keys map to consecutive hashed values, so dense input regions stay dense in the hashed universe. On heavily clustered benchmarks the heaviest bucket reaches $\sim 10^6$ keys, while median and P90 stay in the thousands (Table 6.4, Chapter 6). Worst-case footprint is ≤ 2 MB — cache-resident in L2.

Regardless of distribution, the bucket size is hard-bounded above by the block capacity 2^w, where $w = K - \lfloor \log_2 n \rfloor$ is the suffix length: each bucket physically holds at most 2^w distinct w-bit suffixes. The choice of w is constrained by the overall filter size: the total ARE filter occupies approximately $n \cdot (w + 2)$ bits (w bits of suffix data plus $\sim$2 BPK of metadata). Compared to honest storage of 64-bit keys ($64n$ bits), the filter is meaningfully smaller only for w well below 64. In published range filter evaluations the total per-key budget falls in the 10–20 BPK range, with 14–16 BPK as the typical headline configuration (Table 6.1, Chapter 6). After subtracting the $\sim$2 BPK of rank/select metadata, this corresponds to a suffix width $w \in [12, 15]$ bits in our pipeline. A side benefit of $w = 16$ is that 16-bit suffixes align with SIMD lanes for vector-friendly comparisons.

At $w = 15$ the hard upper bound is $2^{15} = 32\,768$ keys per bucket. On a fully packed bucket of 2^w keys binary search terminates in at most $\log_2(2^w) = w$ iterations — and at $w = 15$ this still costs only $\sim$48 ns, comparable to a single Rank/Select on the metadata bit vector. The bound is distribution-independent and depends only on w, not on n. In real workloads observed buckets are typically two orders of magnitude smaller than this hard limit; the worst case is bounded but rare.

By the way, distributions with dense ranges can be handled by the hybrid filter of Chapter 5: each cluster is isolated into its own exact sub-filter, turning local density into a compression advantage.

Search strategy. We implement both binary search $O(\log k)$ and linear scan $O(k)$ over the k suffixes of a bucket, and pick at query time based on k. Apple M4 Max benchmarks on uniform random w-bit suffixes: linear scan is fastest for small buckets ($\sim$3 ns at $k = 12$, $\sim$5 ns at $k = 24$, ∼10 ns at $k = 64$), while binary search stays cheap even for pathologically large buckets (∼40 ns at $k = 2^{12}$, ∼60 ns at $k = 2^{17}$) — logarithmic growth compresses the worst case to tens of nanoseconds.

The default adaptive dispatch (linear for $k \leq 128$, binary above) gives the best of both branches: near-optimal in the common case and graceful degradation on adversarial distributions.

For uniform 64-bit keys with non-empty queries, the total ERE query takes ∼33–36 ns on Apple M4 Max, stable across $n \in [2^{20}, 2^{26}]$. Bucket search contributes only ∼3 ns at the typical $k \approx 12$; the remainder is Rank/Select navigation on the metadata bit vectors.

2.1.6 Weak Prefix Search

Definition. A *weak prefix query* on a sorted set $S' \subseteq U'$ is specified by a bit string p of length at most $\log_2 |U'|$ and returns the interval of ranks of points in S' whose binary representation has p as a prefix [Goswami et al., 2015, §3.2]. If no such points exist, the answer is arbitrary — hence *weak*. The supporting index answers the query in $O(1)$ time on the word-RAM model.

Use in ERE. Given a query range $[a, b]$ inside a single block, the original construction [Goswami et al., 2015, §3.2] reduces local emptiness to two weak prefix queries. Let p be the longest common prefix of the binary representations of a and b (an $O(1)$ most-significant-bit operation). Then $S' \cap [a, b] \neq \emptyset$ iff at least one of the following holds:

- the largest point in S' prefixed by $p \cdot 0$ exists and is $\geq a$, or
- the smallest point in S' prefixed by $p \cdot 1$ exists and is $\leq b$.

Each side is one WPS query plus a comparison, giving $O(1)$ local queries. The index is realised as a Z-Fast trie keyed by a Monotone Minimal Perfect Hash Function [Belazzougui et al., 2009].

Why we don't use it. The $O(1)$ guarantee is in the word-RAM model where hash table lookups and trie navigation cost $O(1)$ and space is expressed up to $O(n)$ additive terms. The hidden constants dominate in practice.

We implemented three WPS variants, pairing a Z-Fast trie (standard or succinct) with a Monotone Minimal Perfect Hash Function (MMPHF; fast or compressed):

Variant	Space (bits/key)	Query (ns)
Baseline (standard trie + fast MMPHF)	179	1095–1466
Compact (succinct trie + fast MMPHF)	151	372–567
Learned (succinct trie + learned MMPHF)	59	537–762

Table 2.5: Three WPS variants: space and query latency on Apple M4 Max, $L = 64$, $n = 1024$–32768, 64-bit keys. TODO: link to implementation details and full space breakdown.

Even the most compact variant costs 59 BPK — and this is an auxiliary index *on top of* the packed suffix array, which WPS accelerates but does not replace. In our use case (ERE inside an ARE filter, Section 4.1), the suffix width w is typically 7–13 bits per key, so WPS adds 5–8× more space than the suffix data it indexes.

Query latency is equally problematic. WPS replaces only the local search inside a bucket; Rank/Select navigation on the metadata bit vectors (∼27 ns) is identical for both approaches. Table 2.6 compares the bucket-search step alone across representative bucket sizes.

Even on adversarial bucket sizes WPS stays at least 6× slower for the operation it replaces.

Bucket size k	Binary search (ns)	WPS Compact (ns)	Speedup
$\sim$12 (typical, uniform)	5–9	370–567	40–110$\times$
2^{12} ($k \approx 4{,}000$)	$\sim$40	370–567	9–14$\times$
2^{17} ($k \approx 131{,}000$)	$\sim$60	370–567	6–9$\times$

Table 2.6: Bucket-search latency on Apple M4 Max, $L = 64$, 64-bit keys. WPS Compact numbers are from Table 2.5; binary search numbers are from the bucket-scan benchmark of Section 2.1.3. WPS pays a large per-call constant from trie pointer-chasing and hash probing that does not amortise with k, while binary search on packed suffixes scales as $O(\log k)$ with small constants on contiguous memory.

The root cause is that the $O(1)$ theoretical guarantee hides large constants: each "constant-time" trie step involves hashing a variable-length prefix, probing a hash table, and following a pointer — operations that are individually cheap but accumulate over several hash probes per query. For the small buckets that arise in practice ($\leq$13 keys for uniform data, $\leq 2^w$ in general), binary search on packed suffixes is both faster and requires no additional space.

2.1.7 Space Analysis

For K-bit keys, ERE stores $w = K - \lfloor \log_2 n \rfloor$ bits of suffix per key plus $\sim$2.53 BPK of metadata on the one-vector layout (single bit vector D with rank/select indices, Section 2.1.4; physical figure from Table 2.1, stable across n). At $K = 64$ and $n = 2^{30}$ ($w = 34$) this gives $\sim$37 BPK total — a $\sim$42% reduction over honestly storing the full 64-bit keys (64 BPK), but still $\sim$2.5$\times$ above the 14–16 BPK budget of industrial range filters (Table 6.1, Chapter 6).

Space grows linearly with K — unavoidable for an exact structure, since it must store the information distinguishing the keys. This linear dependence motivates the move to approximate range emptiness: by storing hashed fingerprints of length $K \approx \log_2(\mathcal{L}/\varepsilon)$ instead of full keys, the space becomes independent of the original key length.

2.1.8 Complexity Summary

Here W denotes the machine word size.

- **Build:** $O(n \cdot K)$ — a single pass over sorted keys.
- **Query:**

  Expected (uniform data): $O(\frac{K}{W})$ — block index lookup dominates; bucket size is $O(1)$ on average (Section 2.1.5).

  Worst case (all keys collide in a single block): $O(\frac{K}{W} \log n)$.

  Inside ARE with K-bit fingerprints fitting in one machine word ($K \leq W$): reduces to $O(1)$ expected.
- **Space:** $n(K - \log_2 n) + O(n)$ bits, matching the information-theoretic lower bound.

Chapter 3

Key Distributions

3.1 Key Distributions in Practice

The theoretical bounds of Section 1.2.1 hold for any key set. In practice, however, keys in storage systems are not arbitrary — they follow distributions shaped by the application domain. Understanding these distributions is essential for evaluating hash function designs and segmentation strategies in the following chapters.

3.1.1 The Core Assumption

We assume that queries target regions of the key space where stored keys are dense, rather than arbitrary empty regions.

Remark 4. *This holds for most workloads, where queries reflect the application state and therefore concentrate on regions that contain data.*

This assumption has two consequences:

1. FPR in sparse regions of the key space matters less — queries rarely land there.
2. Dense regions dominate the effective FPR experienced by the application.

The SODA hash (Section 4.1) ignores this assumption entirely — it provides worst-case guarantees for any distribution. The truncation hash (Section 4.2) performs well only under uniformity. The hybrid approaches (Chapter 5) explicitly exploit it by isolating dense regions.

3.1.2 Common Industrial Distributions

Keys in LSM-tree storage systems typically exhibit one or more of the following patterns:

Sequential / near-sequential. Auto-incrementing primary keys, monotonic timestamps, log sequence numbers. Keys are approximately evenly spaced with small gaps. The key space is dense — spread is much smaller than the universe.

Clustered. Keys concentrate in a few dense regions separated by large gaps. Examples:

- **User activity:** a social network stores events keyed by user ID prefix + timestamp. Active users produce dense clusters; inactive users leave gaps.

- **Geospatial data:** S2 cell IDs [Google, 2017] — a hierarchical spatial index that maps geographic coordinates to 64-bit integer keys — cluster around populated areas. Rural regions are sparse.
- **Time-partitioned data:** recent partitions are dense (active writes), older partitions are sparse (archived).

High-entropy / uniform-like. UUIDs, cryptographic hashes, randomly generated tokens. Keys are spread across the full key space with no exploitable structure. This is the hardest case for distribution-aware optimizations and the only case where truncation achieves its theoretical BPK.

3.1.3 SOSD Benchmark Datasets

For empirical evaluation, we use real-world datasets from the SOSD (Search on Sorted Data) benchmark [Kipf et al., 2019], masked to 60-bit keys:

Dataset	n (full)	Character
Facebook	200M	User IDs (uint64). Highly clustered — dense regions of sequential IDs with large gaps between ID ranges.
Books	200M	ISBN sequences (uint32). Clustered by publisher prefix, near-sequential within each publisher.
Wiki	200M	Wikipedia edit timestamps (uint64). Moderately clustered — bursts of activity during events, quieter periods between.
OSM	800M	OpenStreetMap cell IDs (uint64). Geospatially clustered — dense in populated areas, sparse in oceans and deserts.

Table 3.1: SOSD datasets used in our evaluation. All exhibit clustering to varying degrees — none are uniform. Benchmarks in Chapter 6 use subsets of these datasets (e.g. $n = 2^{24}$), masked to 60-bit keys.

Note on duplicate keys. We sort and deduplicate every dataset on load. The Facebook, Books and OSM files contain unique keys (no entries dropped), but the Wiki file has many repeated timestamps — multiple edits committed within the same wall-clock second. The duplicate density also grows with offset into the file: empirically, the first 10M entries are $\sim$12% duplicated, the first 50M are $\sim$35%, and the first 134M ($= 2^{27}$) collapse to only $\sim$65M unique keys ($\sim$51% duplicated). For Wiki-based experiments the effective n is therefore smaller than the requested raw count; tables in Chapter 6 report the actual post-dedupe key count.

3.1.4 Synthetic Distributions

To isolate the effect of specific distribution properties, we also generate synthetic key sets:

- **Uniform:** keys drawn uniformly from $[0, 2^{60})$. Average gap $\approx 2^{60}/n$. No clustering. Baseline for truncation-friendly data.
- **Clustered:** a mixture of Gaussian clusters with controlled parameters (number of clusters, intra-cluster spread, inter-cluster gap). Models the typical industrial pattern.

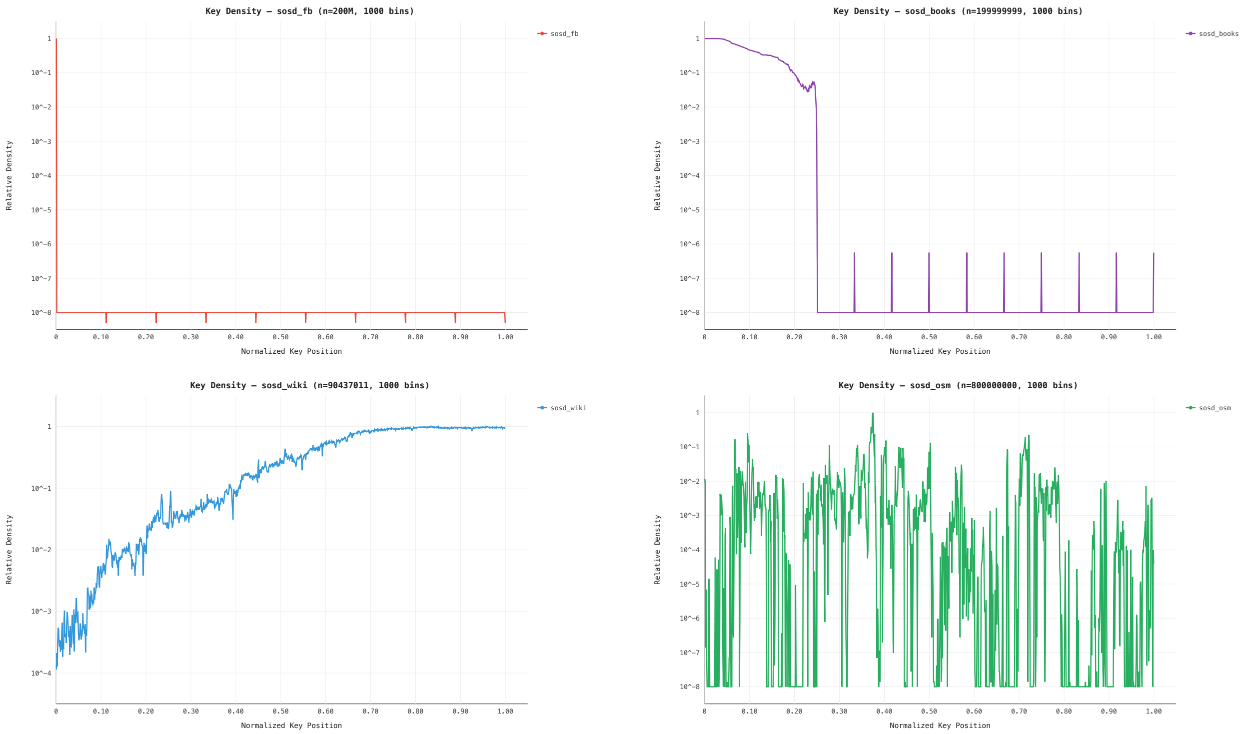

Figure 3.1: Key density histograms for SOSD datasets (1000 bins, log-scale Y-axis). Facebook and Books concentrate keys in a narrow region of the key space. Wiki shows a smooth density gradient. OSM is geospatially clustered with many empty regions.

Together, the SOSD and synthetic datasets span the spectrum from highly clustered (Facebook, Books) through moderately structured (Wiki, OSM) to structureless (uniform random) — allowing us to evaluate each filter design across the range of distributions it may encounter in practice.

Chapter 4

Hash Function Approaches

4.1 Pairwise-Independent Hash (SODA Baseline)

4.1.1 The Hash Function

The paper's original locality-preserving hash [Goswami et al., 2015, §3.1]:

$$h(x) = \big(u(\lfloor x/r \rfloor) + x\big) \bmod r, \quad r = 2^K \tag{4.1}$$

where $u : [U/r] \to [r]$ is drawn from a pairwise-independent family, stored as two 64-bit coefficients (a, b).

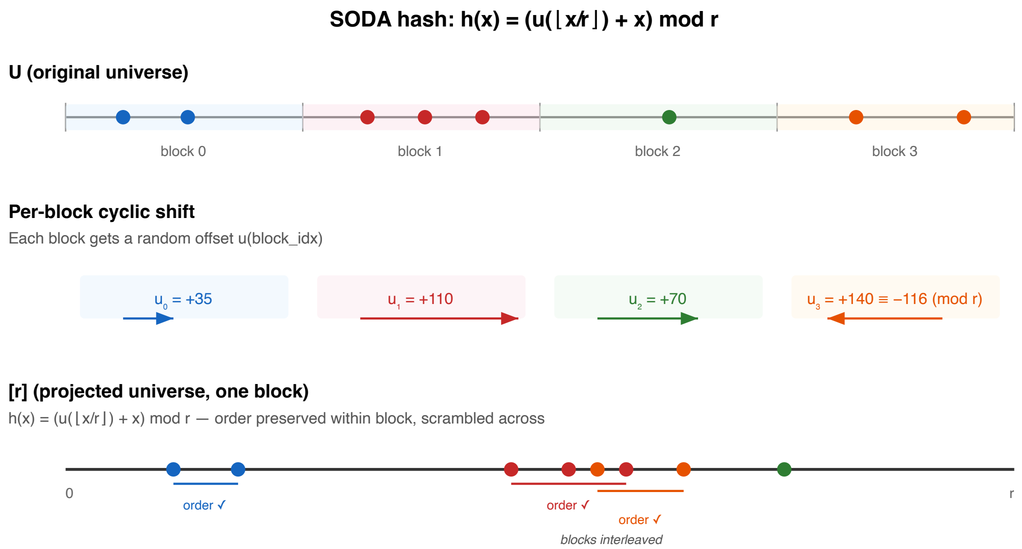

Figure 4.1: SODA hash mechanism: per-block rotation maps keys to $[0, 2^K)$. A query range spanning two blocks produces at most two intervals in the mapped space.

Properties of the hash:

- **Pairwise-independent:** $\Pr[h(x_1) = h(x_2)] \le 1/r$ regardless of key structure. This holds for sequential, clustered, or adversarial data.
- **Locality-preserving:** the hash applies a per-block rotation, so consecutive keys within the same block map to consecutive positions in $[r]$. A query range $[a, b]$ spans at most 2 blocks, producing at most 2 intervals in the mapped space.
- **Compact:** stores only two 64-bit coefficients (a, b) — $O(1)$ bits, independent of n.

4.1.2 ARE Construction With SODA Hash

This is a direct implementation of the construction from Goswami et al. [2015, §3]. The hash design, its proofs of locality-preservation and pairwise independence, and the resulting space/FPR bounds are all from the original paper — our contribution is the practical implementation.

Combining this hash with ERE (Chapter 2) yields the baseline ARE filter:

1. Divide U into blocks of size $r = 2^K$. Let $\text{blockIdx}(x) = \lfloor x/r \rfloor$ be the block index of key x.
2. For each block, compute pairwise-independent shift: top K bits of $(a \cdot \text{blockIdx}(x) + b)$.
3. Hash each key via Equation (4.1).
4. Sort, deduplicate, build ERE over $[0, 2^K)$.
5. Query: hash both endpoints; rotation may split the range into two intervals — check both in ERE.

Filter properties.

- **FPR $\leq \varepsilon$ for any data distribution** — follows from the pairwise independence of the hash.
- **No false negatives** — follows from the locality-preserving property: all images of the query range are checked in ERE.
- **Space:** $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$, giving BPK $= \log_2(\mathcal{L}/\varepsilon) + O(1)$ — matching the information-theoretic lower bound. Including ERE metadata (D_1, D_2), the total cost is $\log_2(\mathcal{L}/\varepsilon) + {\sim}3$ bits per key.

4.1.3 Empirical Validation

On uniform data ($n = 2^{18}$, $\mathcal{L} = 128$), the measured FPR-vs-BPK curve tracks the theoretical bound with a gap of $\sim$3 BPK (ERE overhead from D_1, D_2, Figure 4.2). On non-uniform data with closely spaced keys the gap narrows to $\sim$1 BPK: hash collisions among nearby keys reduce the number of unique fingerprints, making ERE more compact per key (Figure 4.3).

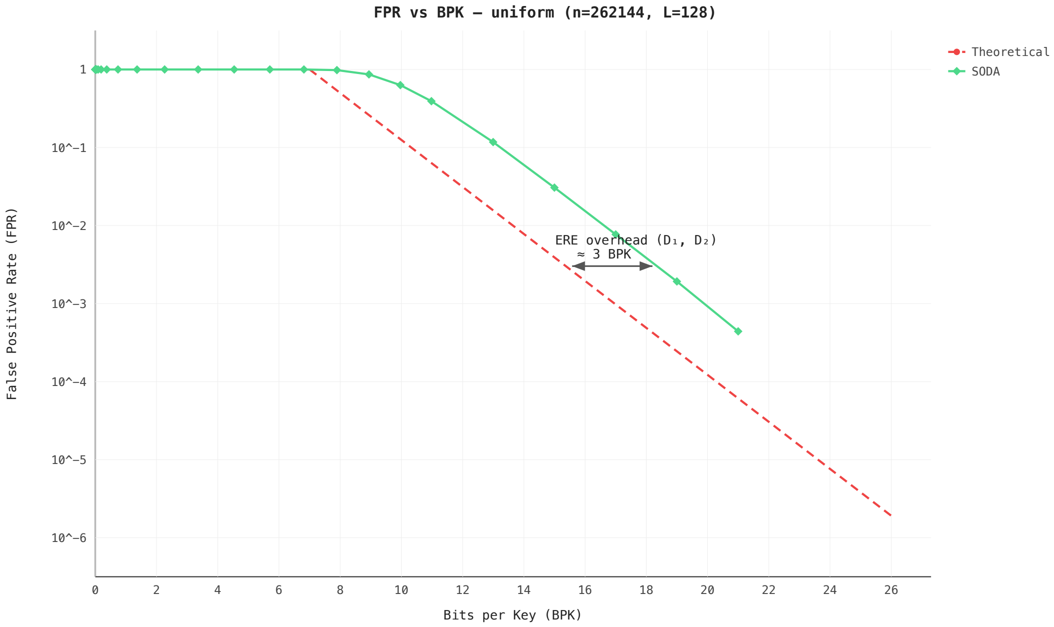

Figure 4.2: FPR vs BPK on uniform keys ($n = 2^{18}$, $\mathcal{L} = 128$). SODA tracks the theoretical bound with $\sim$3 BPK overhead from ERE metadata.

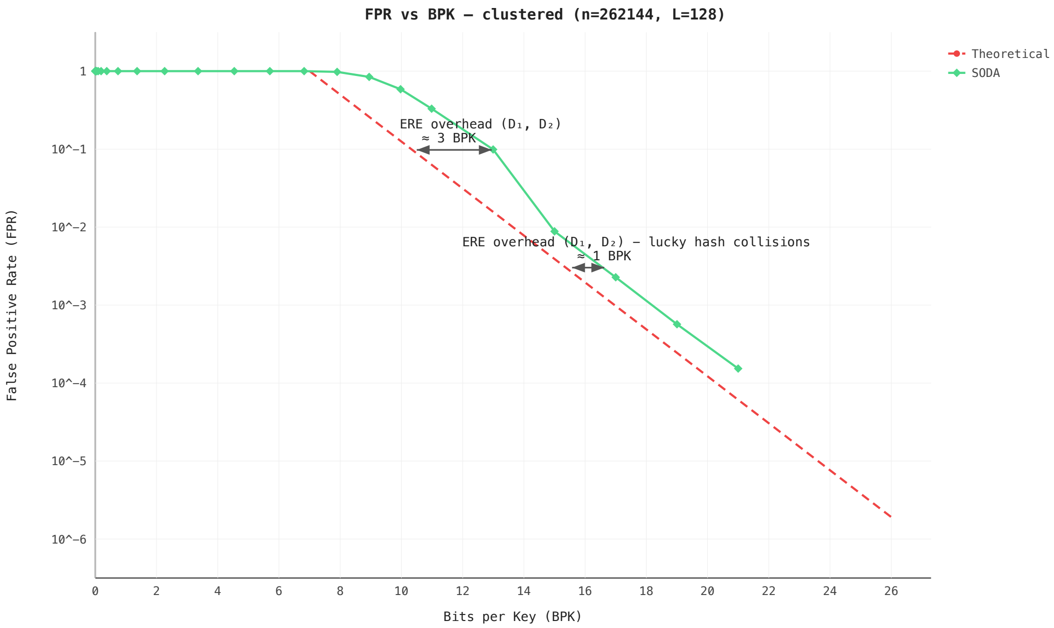

Figure 4.3: FPR vs BPK on clustered keys ($n = 2^{18}$, $\mathcal{L} = 128$). The gap narrows to $\sim$1 BPK as hash collisions among nearby keys reduce the effective number of unique fingerprints.

4.2 Prefix Truncation

4.2.1 The Hash Function

An alternative locality-preserving hash: $h(x) = \lfloor x/2^t \rfloor$, keeping the top $K = L - t$ bits of each key.

Properties of the hash:

- **Locality-preserving:** truncation preserves order — if $a < b$, then $h(a) \leq h(b)$. Intervals map to intervals.
- **Contiguous phantoms:** the 2^t phantom points of each stored key x form an interval of length 2^t in the original universe — all points sharing the same K-bit prefix. This contrasts with the SODA hash, where phantoms are scattered uniformly.
- $O(1)$ **storage:** the hash is a bit-shift — stores only the shift amount t.

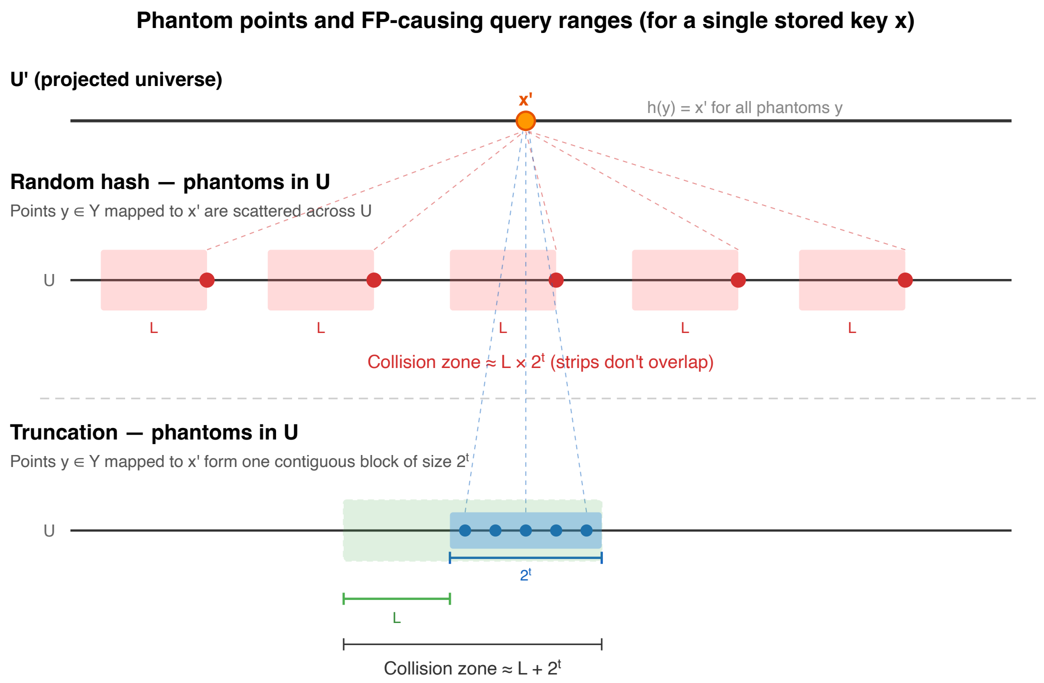

Figure 4.4: Phantom point distribution: random hash (SODA) scatters phantoms uniformly across U; truncation concentrates them in a contiguous block adjacent to the key.

4.2.2 ARE Construction With Truncation

Combining truncation with ERE yields a filter where the contiguous phantom structure (Figure 4.4) changes the collision analysis:

- **SODA hash:** phantoms scattered $\Rightarrow$ collision zone per key is $\mathcal{L} \times 2^t$.
- **Truncation:** phantoms contiguous $\Rightarrow$ collision zone per key is $\mathcal{L} + 2^t$.

The additive (rather than multiplicative) interaction eliminates the $\mathcal{L}$ factor from the filter's space formula:

Hash	Collision zone	K for FPR $\leq \varepsilon$	Filter BPK
Random (SODA)	$\mathcal{L} \times 2^t$	$\log_2(n\mathcal{L}/\varepsilon)$	$\log_2(\mathcal{L}/\varepsilon) + O(1)$
Truncation	$\mathcal{L} + 2^t$	$\log_2(2n/\varepsilon)$	$\log_2(1/\varepsilon) + O(1)$

Table 4.1: Choosing truncation over the SODA hash reduces the resulting filter's BPK by $\log_2(\mathcal{L})$. At $\mathcal{L} = 128$, this is a saving of 7 bits per key.

4.2.3 The Price of Contiguity

The same property that reduces the filter's space is also the fundamental weakness: all false positives are concentrated near stored keys. For a stored key x, its phantom block — the 2^t consecutive points sharing the same K-bit prefix as x — is the only region that can produce false positives.

When inter-key gaps are smaller than the phantom size 2^t, phantom intervals of neighboring keys overlap and FPR approaches 100%. This occurs with:

- **Sequential keys** $(x, x+1, x+2, \ldots)$: every gap is covered by phantoms from both sides.
- **Clustered keys**: phantom intervals overlap, covering the cluster without gaps.

Only uniformly spread keys with gaps $\gg 2^t$ achieve the theoretical FPR — a narrow use case (Figure 4.5). The SODA hash avoids this entirely: pairwise independence scatters phantoms uniformly, giving $\Pr[h(x_1) = h(x_2)] \leq 1/r$ for any distribution, at the cost of $\log_2(\mathcal{L})$ extra bits per key in the resulting filter.

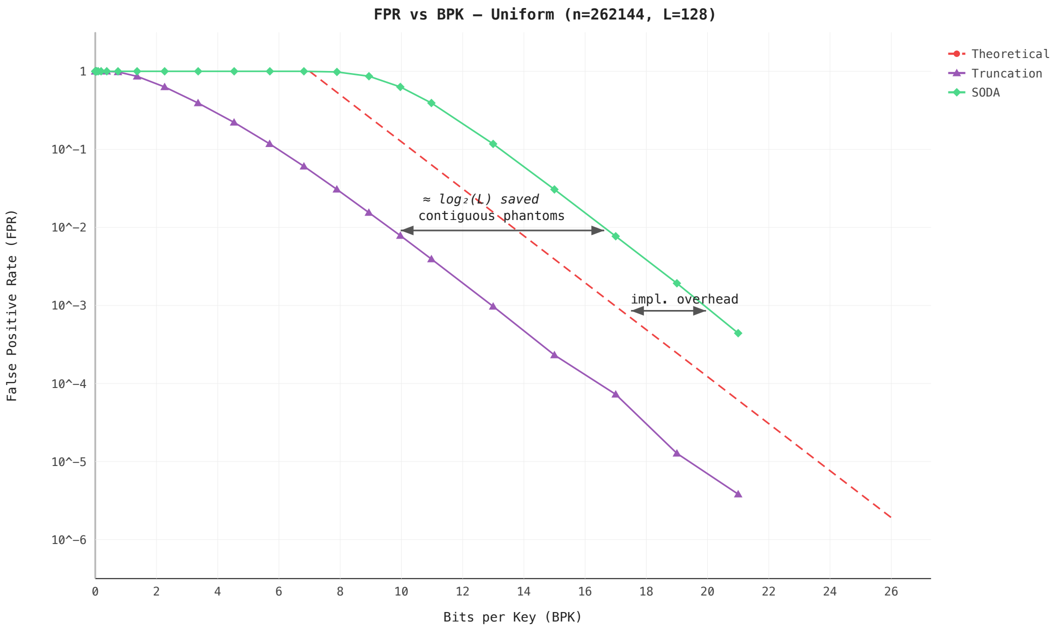

Figure 4.5: FPR vs BPK for truncation vs SODA on uniform keys ($n = 2^{18}$, $\mathcal{L} = 128$). The SODA curve is shifted right by $\approx \log_2(128) = 7$ BPK — matching the theoretical prediction.

4.3 Adaptive Filter

The adaptive filter checks at build time whether hashing is necessary at all.

4.3.1 Exact Mode

1. **Normalize:** subtract $\min(S)$ from all keys, shifting the dataset to start at 0.
2. **Measure spread:** $M = \lceil \log_2(\max(S) - \min(S)) \rceil$.
3. **Decide:**
   - $M \leq K$: the entire dataset fits in K bits $\Rightarrow$ **exact mode**. Build ERE directly over normalized keys. No hash, FPR $= 0$.
   - $M > K$: data too spread $\Rightarrow$ **SODA mode**. Apply pairwise-independent hash, same filter guarantees as Section 4.1.

4.3.2 When Does Exact Mode Trigger?

Exact mode requires $\max(S) - \min(S) < 2^K = n\mathcal{L}/\varepsilon$. Rewriting in terms of density $\rho = (n-1)/(\max(S) - \min(S))$:

$$\rho > \frac{\varepsilon}{\mathcal{L}} \tag{4.2}$$

or equivalently, the average gap $\bar{g} = (\max(S) - \min(S))/(n-1)$ must satisfy $\bar{g} < \mathcal{L}/\varepsilon$.

$\mathcal{L}$	ε	Max avg gap	Min density
128	10^{-3}	128,000	7.8×10^{-6}
128	10^{-6}	1.28×10^{8}	7.8×10^{-9}
16	10^{-3}	16,000	6.25×10^{-5}

Table 4.2: Exact mode threshold for typical parameters. The density requirement is extremely low — exact mode fails only when keys are scattered across a universe much wider than $n \cdot \mathcal{L}/\varepsilon$.

This becomes especially powerful in combination with hybrid segmentation (Chapter 5): each dense cluster is isolated into its own adaptive filter, where the local spread is small enough for exact mode, giving FPR $= 0$ on the dense parts of the data.

4.4 CDF Mapping (Experimental)

An experimental approach: use the empirical CDF of the data as the locality-preserving hash h, redistributing the FPR budget from sparse regions to dense regions.

Status: theoretical exploration. Benchmarks did not show practical improvement over the SODA hash or hybrid segmentation.

4.4.1 Motivation

The information-theoretic lower bound (Theorem 1) holds for all data and query distributions. But if queries follow approximately the same distribution as stored keys — users query ranges where data exists, not empty regions — then FPR in sparse inter-cluster gaps does not matter. If we could *move* FPR budget away from dense regions and into sparse regions, the effective FPR experienced by real queries would be lower for the same number of bits.

4.4.2 The Hash Function

The CDF is a monotonic function that maps keys to a near-uniform distribution (Figure 4.6):

- **Where keys are dense** (clusters): the mapped space is stretched, so the same number of non-keys is spread over a wider interval ⇒ fewer non-keys per unit length ⇒ lower local FPR.

- **Where keys are sparse** (gaps): the mapped space is compressed, non-keys pile up ⇒ higher local FPR. But under our assumption, queries rarely land here.

Monotonicity ($x_1 < x_2 \Rightarrow h(x_1) \leq h(x_2)$) preserves range queries — intervals map to intervals.

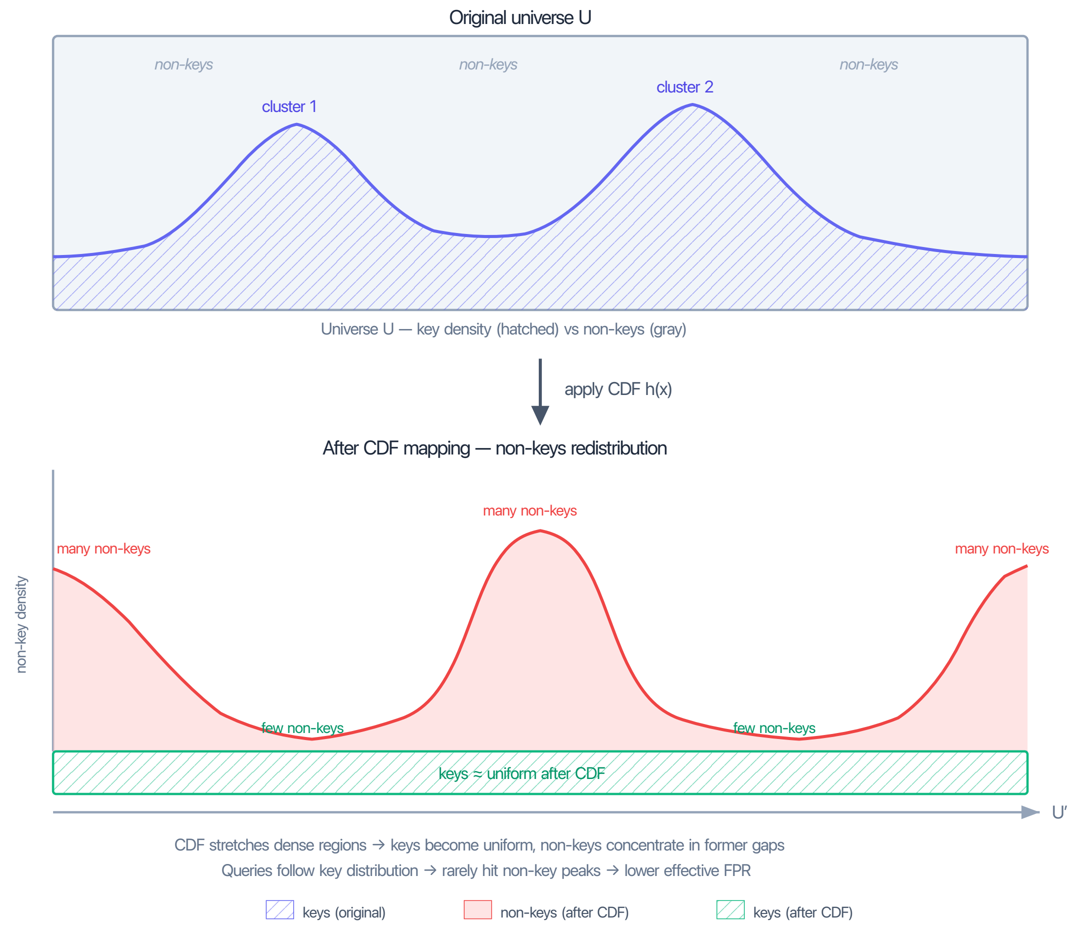

Figure 4.6: CDF redistribution. Top: original universe with non-uniform key density (uniform background + two clusters). After CDF mapping: keys become near-uniform, non-key density inverts — valleys where clusters were (low FPR), peaks where gaps were (high FPR, but queries rarely land there).

Important caveat: because the CDF maps everything monotonically, keys and non-keys are still interleaved in U' — there is no strict separation. The FPR reduction is not formally bounded and depends entirely on the query distribution matching the data distribution.

4.4.3 ARE Construction With CDF Hash

1. Sort input keys.

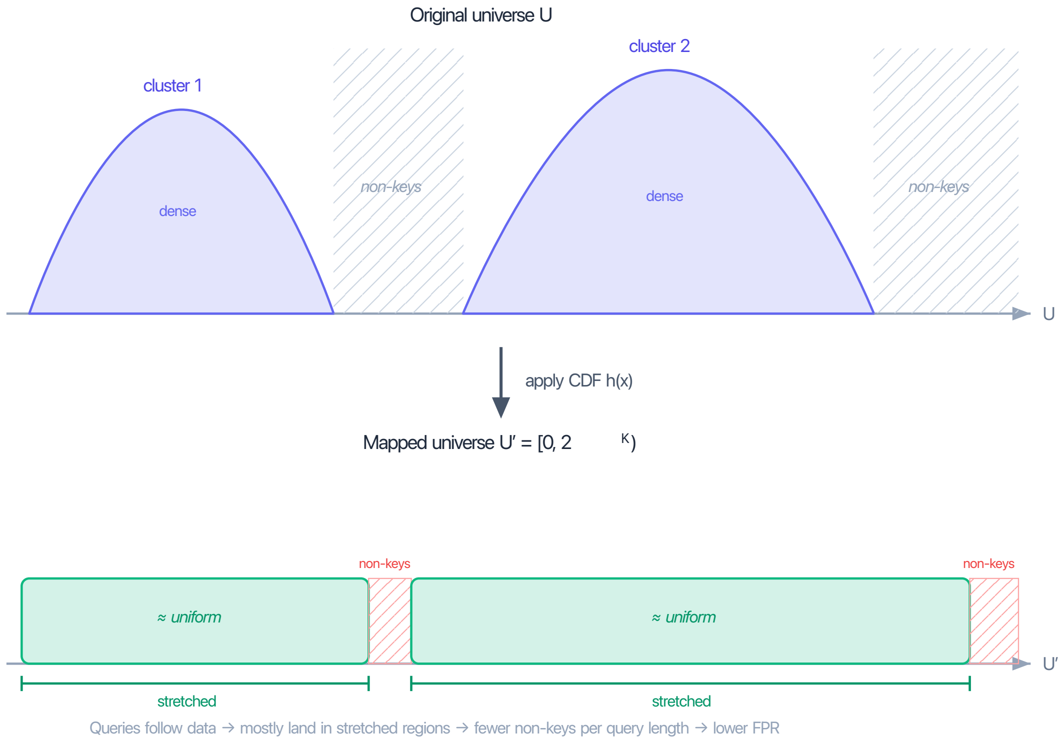

Figure 4.7: Before and after CDF mapping. Dense clusters (top) are stretched into uniform blocks (bottom); sparse gaps are compressed.

2. Build a PGM index [Ferragina and Vinciguerra, 2020] on float64-converted keys to obtain an approximate CDF.
3. Fix monotonicity (PGM positions may be non-monotone due to float64 rounding) via running max.
4. Sample CDF control points every `pgmEpsilon` keys $\Rightarrow$ piecewise-linear model.
5. Compute L_{eff}: the effective range length after CDF distortion. A query of length $\mathcal{L}$ maps to a range of variable width depending on the local CDF slope, bounded by $L_{\text{eff}} = \mathcal{L} \times \max_{\text{segment}} \text{density}$.
6. Set $K = \lceil \log_2(n \cdot L_{\text{eff}}/\varepsilon) \rceil$.
7. Map keys through the piecewise-linear CDF, deduplicate, build ERE.
8. Query: map both endpoints through the same CDF, forward to ERE.

CDF model cost: each control point stores a (key, rank) pair = 128 bits. With `pgmEpsilon` = 128: $\approx n/128$ points ≈ 1 bit per key.

4.4.4 Limitations

- $O(n^2)$ build time due to PGM convex hull construction; constructor returns error for $n > 2^{20}$.
- Float64 precision loss for keys $> 2^{53}$.
- FPR is not formally bounded — depends on query/data distribution match.
- CDF model overhead adds 1–2 BPK on top of ERE storage.

4.4.5 Experimental Results

Benchmarks against the SODA hash showed that CDF-ARE does not provide a consistent practical advantage:

- When query distribution matches data: CDF-ARE uses fewer bits per key, but FPR is comparable or worse due to L_{eff} amplification in dense regions.
- When query distribution diverges from data: FPR degrades catastrophically (40–58% at $\sigma_{\text{query}} = 4 \times \sigma_{\text{data}}$).
- The $O(n^2)$ build cost makes it impractical for large datasets.

Hybrid segmentation (Chapter 5) achieves a similar goal — exploiting clustering — more effectively via segmentation and exact mode, without the CDF overhead.

Chapter 5

Hybrid Segmentation

A single global filter must handle the entire key space — if the spread exceeds 2^K, hashing is unavoidable and FPR is nonzero everywhere. But real-world keys are not spread uniformly: they concentrate in dense clusters separated by large gaps (Section 3.1).

The hybrid idea is to *segment* the key set: isolate each dense cluster into its own sub-filter, where the local spread is small enough for exact mode (FPR = 0, Section 4.3), and route the remaining sparse keys to a fallback filter that uses hashing. Dense regions — where most queries land — pay zero false positives; only the sparse tail incurs FPR $\leq \varepsilon$.

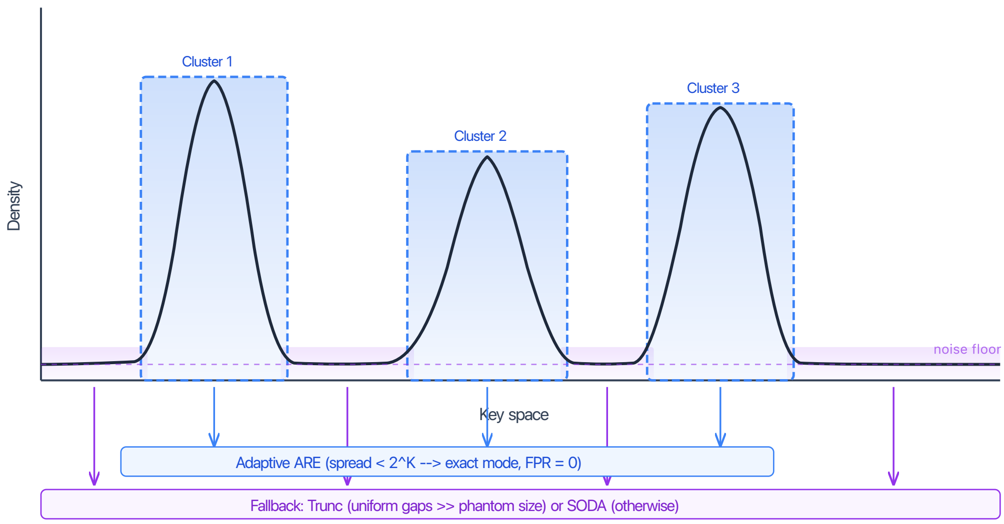

Figure 5.1: Hybrid segmentation overview: dense clusters are isolated into exact-mode sub-filters (FPR = 0), while sparse keys go to a fallback filter (truncation or SODA). The segmentation method differs between approaches — gap-percentile (Section 5.1) vs 1D DBSCAN (Section 5.2) — but the architecture is the same.

Why Clusters Compress So Well

Real-world key distributions concentrate most keys into a few tight regions. In practice, "tight" means many keys share a long common prefix — they differ only in the last few bits. Examples include URLs with shared domain prefixes, file paths with shared directory prefixes, consecutive timestamps, and geospatial S2 cell IDs.

When a cluster is isolated into its own sub-filter, the adaptive filter sees only the local spread $M = \lceil \log_2(\max - \min) \rceil$ of that cluster (Section 4.3). Since $M_{\text{cluster}} \ll K$, exact mode triggers:

no hash, FPR $= 0$, and the universe size shrinks to fit just the cluster's range. The tighter the cluster, the fewer bits the ERE needs.

This is the central advantage over a single global filter: instead of one universe spanning the entire key space (where $M > K$ forces hashing and nonzero FPR), we get many small exact universes — each with zero false positives (Figure 5.2).

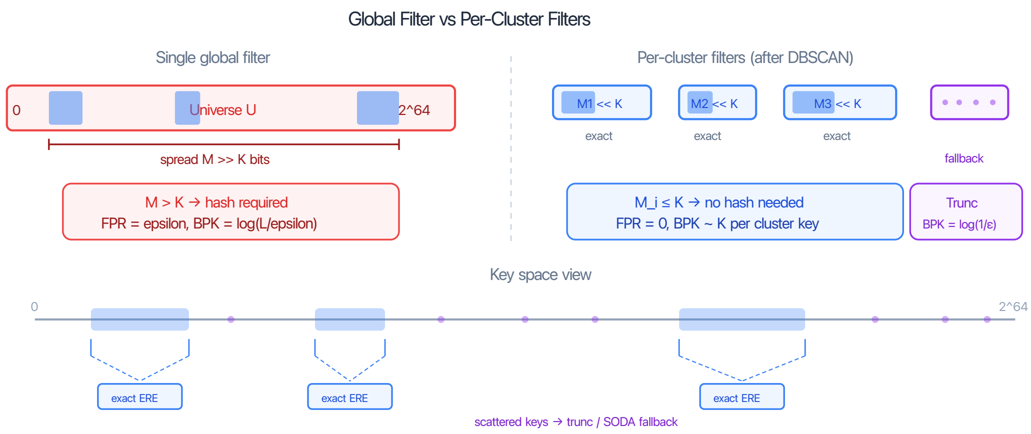

Figure 5.2: Global filter vs per-cluster exact filters. A single global universe forces hashing (nonzero FPR). Isolating each cluster into its own universe enables exact mode (FPR $= 0$) with fewer bits per key.

The Role of Segmentation

The quality of the hybrid filter depends directly on the quality of segmentation — specifically, on how well it separates keys that benefit from exact mode from those that should go to the fallback. Assigning all keys to a single cluster is trivial but useless: the spread is too large for exact mode. Conversely, creating too many small clusters wastes metadata overhead. The segmentation must find the right boundary: tight clusters where exact mode triggers (spread $< 2^K$), while routing sparse outliers to a fallback filter where hashing handles them efficiently.

The rest of this chapter explores two segmentation strategies of increasing sophistication: a simple gap-percentile heuristic (Section 5.1) and a density-based 1D DBSCAN approach (Section 5.2) that addresses its limitations.

5.1 Gap-Percentile Segmentation

The first hybrid approach splits keys into dense clusters and sparse remainder using gap-percentile detection: large gaps between sorted keys mark cluster boundaries.

Given n sorted keys, compute gaps $g_i = \text{key}_{i+1} - \text{key}_i$. The 95th-percentile gap (computed via quickselect) becomes the threshold τ. Then:

1. **Split** at every position where $g_i > \tau$.
2. **Filter:** segments with $\geq 1\%$ of n keys become clusters.
3. **Fallback:** remaining keys go to a truncation filter.

Each cluster is built as an adaptive filter (Section 4.3) — if the cluster's spread fits in K bits, exact mode triggers (FPR $= 0$). Otherwise SODA mode.

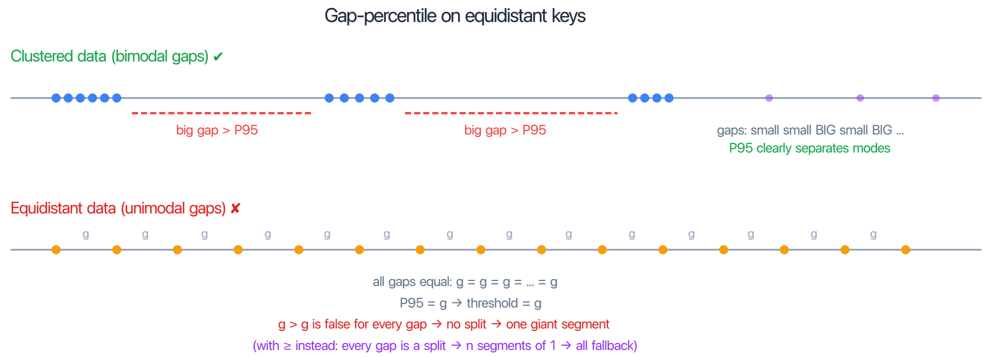

Figure 5.3: Gap-percentile failure on equidistant data: all gaps are equal, so P_{95} equals the common gap and no split points are found.

Limitations.

- **Equidistant data:** when all gaps are equal (sequential keys), the gap distribution has a single mode — there is no "large gap" to split at. The P95 threshold equals the common gap, and strict inequality fails for all gaps, producing one giant segment.
- **No merging:** two adjacent small segments that together would form a valid cluster are never joined.
- **Fallback is always truncation:** if fallback keys have small gaps relative to the phantom size, truncation produces high FPR (Section 4.2.3).

5.2 1D DBSCAN Segmentation (Scan-ARE)

The improved hybrid addresses all three limitations above using density-based clustering.

5.2.1 1D DBSCAN in $O(n)$

DBSCAN [Ester et al., 1996] is normally $O(n^2)$ due to neighbor search. On sorted 1D data, the δ-neighborhood of any point is an interval, so the algorithm reduces to two-pointer sweeps:

1. **Forward sweep:** for each key$[i]$, advance the left pointer while key$[i]$ − key[left] $> \delta$. If the window has $\geq$ `minPts` points, key$[i]$ is a core point.
2. **Reverse sweep:** same logic right-to-left (the forward pass only marks the rightmost points of each dense window).
3. **Core runs:** contiguous core points with gap $\leq \delta$ form cluster cores.
4. **Merge:** adjacent cores within δ of each other are joined.
5. **Border expansion:** non-core points within δ of a core point (*border points*) join the nearest cluster.
6. **Post-filter:** clusters with fewer than `minClusterSize` keys dissolve back to fallback.

Parameter	Value	Role
`minPts`	10	DBSCAN core threshold
`minClusterSize`	256	Post-filter: clusters below this dissolve

Table 5.1: DBSCAN parameters. Two separate parameters — unlike classic DBSCAN which conflates core density and cluster size.

5.2.2 Choosing δ From Problem Parameters

$$\delta = c \cdot \frac{\mathcal{L}}{\varepsilon}, \quad c = 10 \tag{5.1}$$

From the exact mode analysis (Equation 4.2): exact mode triggers when the average gap $< \mathcal{L}/\varepsilon$. So $\delta = 10 \cdot \mathcal{L}/\varepsilon$ means: regions 10× denser than the exact-mode threshold get clustered.

Distribution	Typical gap	Gap vs δ	Result
Cluster (internal gap ∼100)	100	$\ll \delta$	Cluster found
Uniform 64-bit (avg gap $\sim 2^{44}$)	$\sim 10^{13}$	$\gg \delta$	All fallback
Sequential (gap = 1000)	1000	$\ll \delta$	One big cluster

Table 5.2: Sanity check for $\mathcal{L} = 128$, $\varepsilon = 10^{-3}$, $\delta = 1.28 \times 10^6$. No data statistics or percentiles needed — δ is derived purely from $\mathcal{L}$ and ε.

5.2.3 Dual Fallback

Keys not assigned to any cluster go to a fallback filter. The choice between truncation and SODA is made automatically:

1. Let spread = max(fallback keys)−min(fallback keys). Compute phantom size: phantom_size = spread$/2^K$.
2. Compute the 5th-percentile gap (P_5) of fallback keys via quickselect.
3. If $P_5 >$ phantom_size: **truncation** (safe, saves $\log_2(\mathcal{L})$ BPK).
4. If $P_5 \leq$ phantom_size: **adaptive/SODA** (guaranteed FPR).

5.2.4 Query

Clusters are sorted by $[\min, \max]$. For a query $[a, b]$:

1. Binary search over clusters intersecting $[a, b]$ — $O(\log C)$ where C is the number of clusters.
2. Check each intersecting cluster (typically 0–1).
3. Check the fallback filter.
4. Return "empty" only if all sub-filters agree.

5.2.5 FPR Guarantee

Each sub-filter independently targets FPR $\leq \varepsilon$. By a union bound, if a query touches at most C sub-filters, the overall FPR $\leq C\varepsilon$. In practice most queries touch only one sub-filter (either one cluster or just the fallback), so $C = 1$ and FPR $\approx \varepsilon$.

Chapter 6

Experimental Evaluation

6.1 Benchmark Setup

All experiments use 60-bit keys from the SOSD benchmark suite [Kipf et al., 2019]: Facebook user IDs, Wikipedia edit timestamps, OpenStreetMap cell IDs, and Books ISBNs. Synthetic distributions (uniform, clustered) supplement the real-world datasets.

TODO: 60 bits - SNARF overflow when values are close to UINT64_MAX, measure separatly

Competing implementations:

- **Grafite** [Costa et al., 2024]: information-theoretically optimal range filter (SIGMOD 2024).
- **SNARF** [Vaidya et al., 2022]: learned CDF-based range filter.
- **SuRF** [Zhang et al., 2018]: succinct range filter with three variants (base, hash suffix, real suffix).
- **Bloom ARE**: trivial baseline — standard Bloom filter with $\mathcal{L}$ point lookups per range query.

All filters are evaluated on FPR-vs-BPK tradeoff curves, measured over 10^6 random empty-range queries per configuration — enough to register 10^3–10^4 false positives at our target FPR range of 10^{-2}–10^{-3} (Section 6.2, "FPR targets").

6.2 Workload Parameter Choices

Range filters in LSM-tree storage systems are built *per SSTable*: each on-disk file carries its own filter, and the relevant input size n is the number of keys in one SSTable, not the database total. Production deployments keep SSTables small — tens to hundreds of megabytes — for several reasons listed below.

Why SSTables stay small.

1. **Memory and time, compounded.** The build of the filter during compaction operations needs random access to keys, so the working set — not just the filter — must be held in RAM during the operation. Filter construction itself runs at a few million keys per second: Costa et al. [2024] report $\sim$7 M keys/s for Grafite at $n = 2 \times 10^8$ on a single thread. Both RAM and time scale linearly with n; compacting an $n = 2^{30}$ SSTable claims gigabytes of RAM and minutes of CPU on a single operation, while the foreground service runs on what is left.

2. **Cloud I/O.** Loading and saving large SSTables is bandwidth-bound on object storage backends (S3, GCS): a multi-gigabyte file streamed to a remote VM takes minutes even on a fast link.

3. **Damage radius.** Corruption of one SSTable file or one object loses exactly the keys in that file. Smaller files bound the blast.

4. **Sharding.** Multiple smaller SSTables can be queried in parallel for the on-disk scans following a filter probe, migrated independently between nodes, and replicated at file granularity; one giant file cannot. TODO: do we really need this point?

Realistic per-SSTable n. The default RocksDB `target_file_size_base` is 64 MB, with production deployments commonly tuning to 128 MB; entry sizes in real workloads (key + value) are roughly 60–150 bytes on average [Cao et al., 2020], putting the typical per-SSTable count at $n \in [2^{19},\ 2^{21}]$.

Two production cases push n higher. Secondary-index and counter column families (CFs — RocksDB's per-table partitioning, with independent memtables and SSTables) store all information in the key and use the value as a placeholder: in Facebook's social-graph workload (UDB), the `Assoc_count` CF has 20-byte bigint keys with values under 8 B, averaging ~ 28 B/entry, and similar patterns appear in other index/counter CFs [Cao et al., 2020]. KV-separation deployments (WiscKey [Lu et al., 2016], RocksDB BlobDB, Pebble) store only a key plus a 10–20-byte blob handle in the SSTable, with values placed in separate blob files. In both cases the per-SSTable count rises to $n \in [2^{22},\ 2^{23}]$.

Range filter papers [Zhang et al., 2018, Luo et al., 2020, Vaidya et al., 2022, Eslami Beni et al., 2024, Costa et al., 2024] report headline numbers at $n \in [2^{24},\ 2^{28}]$, but this is a single-filter microbenchmark convention rather than a deployment claim: SuRF, SNARF, and Rosetta integrate per-SSTable into RocksDB; Memento targets B-Trees (WiredTiger); Grafite is presented as a stand-alone structure. The SOSD benchmark [Kipf et al., 2019] provides datasets at this same scale (200M keys $\approx 2^{27.6}$).

Why not a global filter. A database-wide filter is faster on the query side: range filter query time is essentially independent of n, so each per-file probe is constant-time — but a per-file design pays one such probe per SSTable, while a single global filter needs only one. The update side is the opposite story: LSM compaction atomically replaces SSTables, making per-file filter rebuild touch only the keys in that compaction, while a global filter must either rebuild over all keys or maintain counting structures that support deletion. GRF [Wang et al., 2024], a recent global range filter for LSM-trees, achieves single-probe queries through shape encoding; production systems — and our benchmarks — still adopt the simpler per-SSTable model.

The query-side argument is also weaker than it first looks. Within a level (L1+), RocksDB stores SSTables as a non-overlapping *sorted run*: each file carries [*smallest*, *largest*] key metadata in the MANIFEST, and a query does an $O(\log \#\mathit{files})$ binary search on this metadata to identify only the SSTables whose key range overlaps the query, then probes their per-file filters [Facebook, 2024b].

Our choice of n. We benchmark at three anchor scales:

- $n = 2^{20}$ — per-SSTable production workload;
- $n = 2^{24}$ — large SSTable or aggregate filter;
- $n = 2^{28}$ — paper-evaluation and SOSD scale.

$n = 2^{30}$ is additionally reported as an asymptotic stretch point, beyond realistic per-SSTable deployments.

Per-key budget. Published range filter evaluations consistently report total per-key budgets in the 10–20 BPK range, with 14–16 BPK as the typical headline configuration (Table 6.1). The LSM point-filter convention sits at the lower end of this window: RocksDB ships Bloom filters at 10 BPK by default [Facebook, 2024a], reported by Dayan et al. [2017] as the cross-system standard. We adopt the same target range.

Filter	BPK budget	Reported FPR	Notes
SuRF [Zhang et al., 2018]	10–16	$\sim 10^{-3}$ at 16 BPK*	RocksDB-anchored
Rosetta [Luo et al., 2020]	10–20	10^{-2} to 10^{-4}	FPR-vs-BPK sweep
SNARF [Vaidya et al., 2022]	16	$6.2 \times 10^{-5*}$	Headline result
Memento [Eslami Beni et al., 2024]	≥ 12	ε-parameterised	Design minimum
Grafite [Costa et al., 2024]	$\sim$12	$\leq \mathcal{L}/2^{\mathrm{BPK}-2}$	Adversarial bound

Table 6.1: Per-key memory budgets and corresponding false positive rates reported in recent range filter literature. FPR depends strongly on range length $\mathcal{L}$ and workload. *SuRF and SNARF FPR values at 16 BPK are both reported in Vaidya et al. [2022] for direct comparison.

FPR targets. We report, for each filter, the BPK required to reach two fixed FPR targets: 10^{-2} and 10^{-3}. Both targets sit inside the operating range of published range filter evaluations: classical filters [Zhang et al., 2018, Luo et al., 2020, Wang et al., 2024] anchor their headline numbers around 10^{-2} to 10^{-3}, while modern optimal filters [Vaidya et al., 2022, Costa et al., 2024, Eslami Beni et al., 2024] sweep deeper. They also align with the LSM point-filter convention: Cassandra's default `bloom_filter_fp_chance` is 0.01 for all compaction strategies except LCS [Apache Software Foundation, 2024].

Query range length $\mathcal{L}$. We sweep $\mathcal{L} \in \{1, 16, 128, 1024, 4096, 16384, 65536\}$. The lower half is production-aligned: in Facebook's UDB workload, more than 60% of iterator operations read exactly one key, the P90 is below 100 keys, and scans above 10,000 are very rare [Cao et al., 2020]; YCSB Workload E caps short-range scans at 100. Range filter papers [Vaidya et al., 2022, Luo et al., 2020, Costa et al., 2024, Eslami Beni et al., 2024] report headline FPR in this same window: SNARF at $\mathcal{L} = 256$, Rosetta at $\mathcal{L} = 64$, Grafite and Memento sweeping $\mathcal{L} \in \{1, 32, 1024\}$. The upper half ($\mathcal{L} \geq 4096$) extends past production P99 and any reported literature sweep. Such large $\mathcal{L}$ values are usually excluded from filter benchmarks because hashing-based filters cannot maintain low FPR there; we include them because our construction can.

6.3 ERE Backend Comparison: Two-Vector vs One-Vector

We compare the two-vector and one-vector ERE backends as the exact layer of two ARE wrappers (SODA and Greedy+Merge) across six distributions. The analytic prediction from Section 2.1.4 is $\Delta = M/n$ BPK, where M is the number of non-empty blocks; the maximum is $B/n = 1$ BPK and the Poisson middle for uniform input at $\lambda = 1$ is $1 - e^{-1} \approx 0.632$ BPK. The cross-distribution numbers below confirm the prediction end-to-end.

Dataset	SODA two-vec BPK	SODA one-vec BPK	SODA Δ	Greedy+Merge two-vec BPK	Greedy+Merge one-vec BPK	Greedy+Merge Δ
`uniform`	21.632	21.000	0.632	21.632	21.000	0.632
`clustered`	21.110	21.000	0.110	17.201	16.961	0.240
`sosd_fb`	21.001	21.000	0.001	11.232	11.000	0.232
`sosd_wiki`	21.530	21.530	0.000	9.708	9.530	0.177
`sosd_osm`	21.930	21.500	0.430	32.817	32.525	0.293
`sosd_books`	21.000	21.000	0.000	4.517	4.000	0.517
Average	21.367	21.172	0.196	16.184	15.836	0.349

Table 6.2: Total ARE-level BPK with the two-vector (D_1, D_2) ERE backend (Section 2.1.2) versus the one-vector D backend (Section 2.1.4). Workload: $n = 2^{20}$ (or all available keys, whichever is smaller), $\mathcal{L} = 4096$, $\varepsilon = 0.01$.

Reading the table. SODA's locality-preserving hash keeps clustered inputs clustered in the hashed universe, so the SOSD datasets leave most blocks empty under SODA ($\Delta \leq 0.001$ BPK on `sosd_fb`, `sosd_wiki`, `sosd_books`); only `uniform` attains the Poisson value. Greedy+Merge isolates dense regions and builds a separate ERE per cluster; even within a cluster the keys retain sub-structure, so M/B stays well below 1. Its absolute BPK is dominated by the wrapper rather than ERE, so even small M/n values translate into a visible relative improvement (e.g. `sosd_books` drops from 4.517 to 4.000 BPK).

Query latency. Table 6.3 reports average ARE query latency on the same workload. SODA gains uniformly (1.03–1.88×); Greedy+Merge regresses on `sosd_fb` (0.78×) and `sosd_wiki` (0.80×), where the input collapses into a single cluster and most queries short-circuit before reaching the bitvectors — the one-vector overhead is paid without the cache-locality benefit. Both regressions are small in absolute terms ($\leq$ 70 ns) and the cross-distribution average favours the one-vector backend in both wrappers.

Dataset	SODA two-vec ns	SODA one-vec ns	SODA speedup	Greedy+Merge two-vec ns	Greedy+Merge one-vec ns	Greedy+Merge speedup
`uniform`	156.2	140.5	1.11×	199.7	170.0	1.18×
`clustered`	370.0	314.2	1.18×	523.3	389.8	1.34×
`sosd_fb`	703.0	619.0	1.14×	251.1	322.7	0.78×
`sosd_wiki`	562.6	533.7	1.05×	276.4	346.9	0.80×
`sosd_osm`	306.2	162.4	1.88×	453.1	427.9	1.06×
`sosd_books`	656.7	635.2	1.03×	315.9	209.0	1.51×
Average	**459.1**	**400.8**	**1.15×**	**336.6**	**311.0**	**1.08×**

Table 6.3: Average ARE query latency, two-vector vs one-vector ERE backend. Same workload as Table 6.2. Apple M4 Max, 32,768 mixed queries, 3 rounds. Average is over per-distribution means. TODO: n is what, L is what?, key size?

6.4 ERE Bucket Occupancy on SOSD

To characterize how the SODA hash distributes input keys across ERE blocks on real workloads, we measured per-bucket statistics for each SOSD dataset across three range lengths.

Dataset	n	$\mathcal{L}$	w	X_{50}	X_{90}	$\max_b k_b$	Footprint	Fill
`sosd_fb`	2^{27}	16	11	27	103	463	637 B	22.6%
`sosd_fb`	2^{27}	256	15	208	545	1,589	2.9 KB	4.9%
`sosd_fb`	2^{27}	4096	19	1,759	2,947	6,176	14.3 KB	1.2%
`sosd_wiki`	65M*	16	12	3,240	3,843	4,054	5.9 KB	**99.0%**
`sosd_wiki`	65M*	256	16	52,565	58,855	62,037	121 KB	**94.7%**
`sosd_wiki`	65M*	4096	20	847,135	938,217	956,498	2.3 MB	**91.2%**
`sosd_books`	2^{27}	16	11	812	875	978	1.3 KB	47.8%
`sosd_books`	2^{27}	256	15	13,215	13,670	14,039	25.7 KB	42.8%
`sosd_books`	2^{27}	4096	19	211,319	217,641	218,469	506 KB	41.7%
`sosd_osm`	2^{29}	16	11	3	5	35	48 B	1.7%
`sosd_osm`	2^{29}	256	15	3	5	85	159 B	0.3%
`sosd_osm`	2^{29}	4096	19	3	5	502	1.2 KB	0.1%

Table 6.4: Bucket-size statistics for the inner ERE of SodaARE on the SOSD datasets, at $\varepsilon = 0.01$. *`sosd_wiki` effective n after deduplication of repeated timestamps (Section 3.1.3).

Columns.

w — Suffix length in bits, $w = K - \lfloor \log_2 n \rfloor$, with $K = \lceil \log_2(n\mathcal{L}/\varepsilon) \rceil$. Physical block capacity is 2^w distinct w-bit suffixes.

X_{50}, X_{90} — Key-weighted CDF percentiles over non-empty buckets: the smallest bucket size such that buckets of size $\leq X_p$ collectively hold at least $p \cdot n$ keys.

$\max_b k_b$ — Empirical maximum bucket size.

Footprint — Bit-packed size of the heaviest bucket ($\max_b k_b \cdot w/8$ bytes).

Fill — $\max_b k_b / 2^w$ — the heaviest bucket's saturation against physical capacity.

Observations. `sosd_fb` and `sosd_osm` stay well below saturation at every $\mathcal{L}$. `sosd_books` runs at ∼42% across $\mathcal{L}$. `sosd_wiki` fully saturates at $\mathcal{L} = 16$ (99%). All footprints fit L2 cache; the lighter cases fit L1.

When fill approaches 100%, the SODA hash has run out of resolution: every key in a dense input region collapses into a single ARE block, and the bucket-search cost for those queries becomes $O(2^w)$ rather than $O(1)$ on average. For `sosd_wiki` this means binary search on a fully packed bucket of up to $\sim 10^6$ keys — still bounded by w iterations and cache-resident in L2 (Section 2.1.5), but the buckets are no longer lightly loaded as the SODA construction assumes.

6.5 Large-Range Performance: $\mathcal{L} = 65536$

The combination of truncation-based hashing and exact-mode clusters enables handling very large range queries that would be impractical for other approaches.

At $\mathcal{L} = 65536$, non-hybrid filters face a fundamental problem: the fingerprint length $K = \lceil \log_2(n \cdot \mathcal{L}/\varepsilon) \rceil$ must grow by $\log_2(\mathcal{L}) = 16$ bits to maintain the same FPR. For SODA, this means 16 extra BPK. For Bloom, $\mathcal{L} = 65536$ point lookups per query is impractical.

Scan-ARE sidesteps this on two levels:

Exact clusters. Each cluster stores its $[\min, \max]$ bounds. If a query range $[a, b]$ overlaps a cluster's bounds, at least one stored key is inside $[a, b]$ — that is a true positive, not a false positive. If $[a, b]$ does not overlap any cluster, the cluster contributes nothing. If $[a, b]$ falls entirely within a cluster, the sub-filter is exact (Section 4.3), so false positives are impossible. Either way, exact clusters produce zero false positives regardless of $\mathcal{L}$.

Truncation fallback. The sparse fallback uses truncation hashing (Section 4.2), where phantom points form a block adjacent to each stored key (Figure 4.4). A false positive can only occur when a query *endpoint* a or b lands inside a phantom block — if a stored key lies inside $[a, b]$, that is a true positive, not a false positive. Because phantoms do not scatter across the universe the way SODA phantoms do, the FPR of truncation is independent of the query range length $\mathcal{L}$. The fallback thus avoids the $\log_2(\mathcal{L})$ BPK penalty that SODA-based filters must pay.

6.5.1 SOSD Results ($n = 2^{24}$, $\mathcal{L} = 65536$)

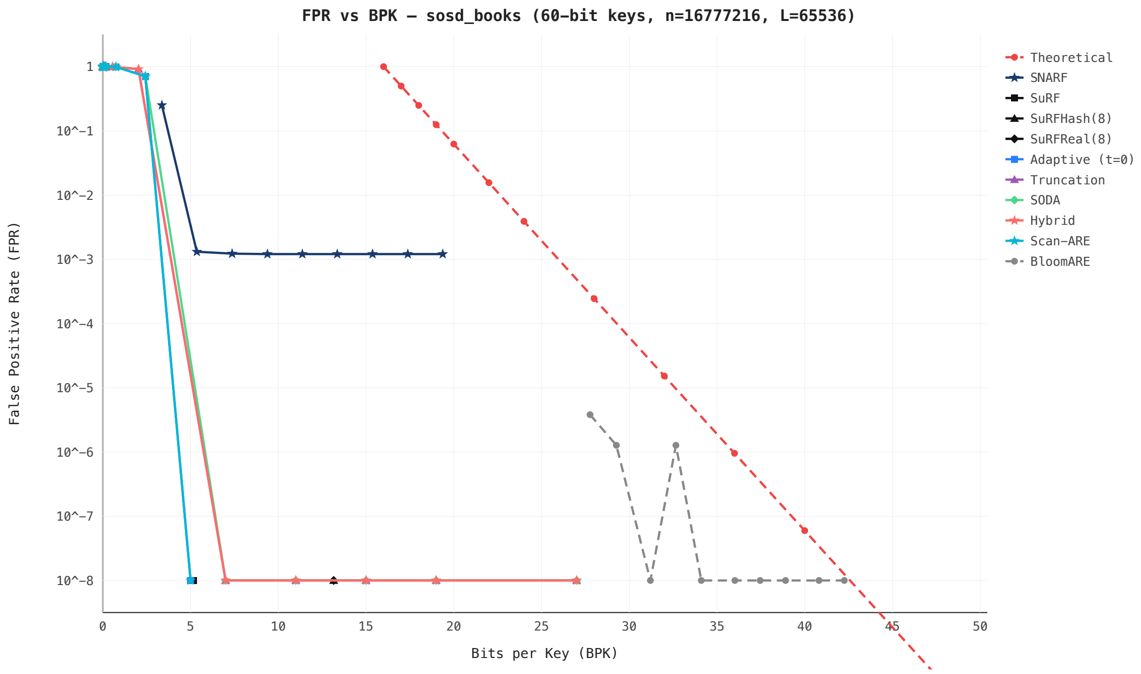

Figure 6.1: FPR vs BPK — SOSD Books (ISBN sequences, highly clustered), $n = 2^{24}$, $\mathcal{L} = 65536$. Scan-ARE achieves FPR $< 10^{-8}$ at ∼5 BPK; all non-hybrid filters degrade to FPR ≈ 1.

On both datasets, Scan-ARE achieves FPR $< 10^{-8}$ at ∼5 BPK, while Truncation, SODA, and Bloom all degrade to FPR ≈ 1 at comparable budgets. SNARF plateaus at FPR $\sim 10^{-3}$. Only Grafite remains competitive on Wiki (∼10 BPK for FPR 10^{-2}), but at 2–3× the space of Scan-ARE.

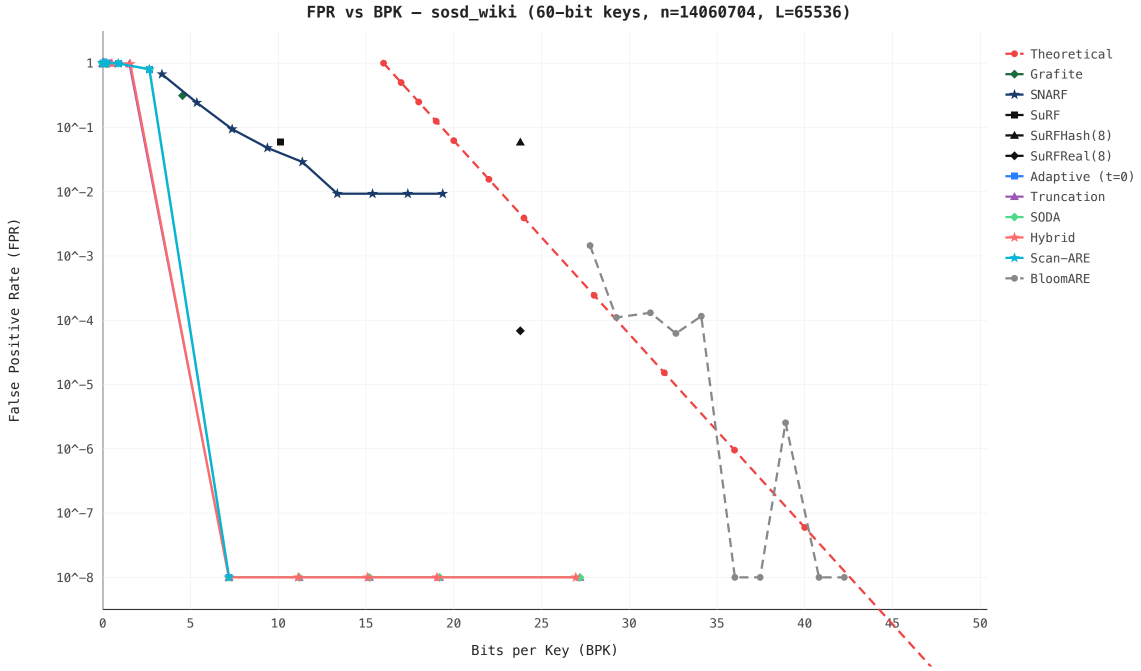

Figure 6.2: FPR vs BPK — SOSD Wiki (Wikipedia timestamps, moderately clustered), $n = 2^{24}$, $\mathcal{L} = 65536$. Scan-ARE dominates; only Grafite remains competitive at ∼10 BPK but with 2–3× more space.

6.6 BPK at Industry-Standard FPR Targets

In practice, storage systems target a specific FPR — typically 10^{-2} or 10^{-3} — and want to minimize the filter's memory footprint. The relevant question is: how many bits per key does each filter need to reach the target FPR?

We report this metric separately for each of the 9 benchmark distributions: 4 real-world (SOSD) and 5 synthetic. Each distribution exercises different filter strengths — a filter that excels on clustered data may be mediocre on uniform, and vice versa.

SOSD Distributions

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.5: BPK to reach target FPR — SOSD Facebook ($n = 2^{24}$, highly clustered).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.6: BPK to reach target FPR — SOSD Books ($n = 2^{24}$, clustered by publisher).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.7: BPK to reach target FPR — SOSD Wiki ($n = 2^{24}$, moderately clustered).

Synthetic Distributions

6.7 Summary of Results

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.8: BPK to reach target FPR — SOSD OSM ($n = 2^{24}$, geospatially clustered).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.9: BPK to reach target FPR — Clustered synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.10: BPK to reach target FPR — Uniform synthetic ($n = 2^{24}$).

	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
Filter	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.11: BPK to reach target FPR — Spread synthetic ($n = 2^{24}$).

Filter	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.12: BPK to reach target FPR — Zipfian synthetic ($n = 2^{24}$).

Filter	$\mathcal{L} = 128$		$\mathcal{L} = 65536$	
	FPR 10^{-2}	FPR 10^{-3}	FPR 10^{-2}	FPR 10^{-3}
Scan-ARE	—	—	—	—
Grafite	—	—	—	—
SNARF	—	—	—	—
SODA	—	—	—	—
Truncation	—	—	—	—
SuRF-Real	—	—	—	—
Bloom	—	—	—	—

Table 6.13: BPK to reach target FPR — Temporal synthetic ($n = 2^{24}$).

Filter	BPK ($\mathcal{L}$=65536)	FPR	Distribution dependence
Scan-ARE	$\sim$5	$< 10^{-8}$	Best on clustered data
Grafite	$\sim$10	$\sim 10^{-2}$	Distribution-independent
SNARF	$\sim$28	$\sim 10^{-3}$	Learned, data-dependent
SODA	$\sim$5	≈ 1 (degenerate)	Distribution-independent
Truncation	$\sim$3	≈ 1 (degenerate)	Fails on clustered data
Bloom	$\sim$30	$\sim 10^{-4}$	Distribution-independent

Table 6.14: Comparison at $n = 2^{24}$, $\mathcal{L} = 65536$ on SOSD Books. Scan-ARE dominates the Pareto frontier by exploiting cluster density for exact-mode sub-filters.

Chapter 7

Conclusion

Bibliography

Apache Software Foundation. Apache Cassandra documentation: Bloom filters, 2024. URL `https://cassandra.apache.org/doc/latest/cassandra/managing/operating/bloom_filters.html`. Accessed: 2026-04-26.

Djamal Belazzougui, Paolo Boldi, Rasmus Pagh, and Sebastiano Vigna. Monotone minimal perfect hashing: Searching a sorted table with O(1) accesses. In *Proceedings of the Twentieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '09)*, pages 785–794. SIAM, 2009.

Zhichao Cao, Siying Dong, Sagar Vemuri, and David H. C. Du. Characterizing, modeling, and benchmarking RocksDB key-value workloads at Facebook. In *18th USENIX Conference on File and Storage Technologies (FAST '20)*, pages 209–223. USENIX Association, 2020.

Marco Costa, Paolo Ferragina, and Giorgio Vinciguerra. Grafite: Taming adversarial queries with optimal range filters. In *Proceedings of the 2024 International Conference on Management of Data (SIGMOD '24)*. ACM, 2024. doi: 10.1145/3639258.

Niv Dayan, Manos Athanassoulis, and Stratos Idreos. Monkey: Optimal navigable key-value store. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17)*, pages 79–94. ACM, 2017. doi: 10.1145/3035918.3064054.

Navid Eslami Beni, Ioana O. Bercea, and Niv Dayan. Memento filter: A fast, dynamic, and robust range filter. In *Proceedings of the 2024 International Conference on Management of Data (SIGMOD '24)*. ACM, 2024. doi: 10.1145/3698820.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96)*, pages 226–231, 1996.

Facebook. RocksDB Bloom filter, 2024a. URL `https://github.com/facebook/rocksdb/wiki/RocksDB-Bloom-Filter`. Accessed: 2026-04-26.

Facebook. RocksDB leveled compaction, 2024b. URL `https://github.com/facebook/rocksdb/wiki/Leveled-Compaction`. Accessed: 2026-04-26.

Paolo Ferragina and Giorgio Vinciguerra. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proceedings of the VLDB Endowment*, 13(8):1162–1175, 2020.

Google. S2 Geometry Library, 2017. URL `https://s2geometry.io`. Accessed: 2026-03-25.

Mayank Goswami, Allan Grønlund Jørgensen, Kasper Green Larsen, and Rasmus Pagh. Approximate range emptiness in constant time and optimal space. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '15)*, pages 848–858. SIAM, 2015.

Andreas Kipf, Ryan Marcus, Alexander van Renen, Mihail Stoian, Alfons Kemper, Tim Kraska, and Thomas Neumann. SOSD: A benchmark for learned indexes. *NeurIPS Workshop on Machine Learning for Systems*, 2019. URL `https://arxiv.org/abs/1911.13014`.

Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. WiscKey: Separating keys from values in SSD-conscious storage. In *14th USENIX Conference on File and Storage Technologies (FAST '16)*, pages 133–148. USENIX Association, 2016.

Siqiang Luo, Subhadeep Chatterjee, Rafael Ketsetsidis, Niv Dayan, Wilson Qin, and Stratos Idreos. Rosetta: A robust space-time optimized range filter for key-value stores. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*, pages 2071–2086. ACM, 2020.

Michael Mitzenmacher and Eli Upfal. *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Structures*. Cambridge University Press, 2 edition, 2017.

Kapil Vaidya, Subhadeep Chatterjee, Eric Knorr, Michael Mitzenmacher, Stratos Idreos, and Tim Kraska. SNARF: A learning-enhanced range filter. *Proceedings of the VLDB Endowment*, 15(8):1632–1644, 2022.

Hengrui Wang, Te Guo, Junzhao Yang, and Huanchen Zhang. GRF: A global range filter for LSM-trees with shape encoding. *Proceedings of the ACM on Management of Data*, 2(3), 2024. doi: 10.1145/3654944.

Huanchen Zhang, Hyeontaek Lim, Viktor Leis, David G Andersen, Michael Kaminsky, Kimberly Keeton, and Andrew Pavlo. SuRF: Practical range query filtering with fast succinct tries. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*, pages 323–336. ACM, 2018.